School of Computing and Mathematical Sciences

CO7201 Individual Project

Preliminary Report

Software Bug Tracking & Reporting Tool

Parth Maru

pbm5@student.le.ac.uk

Student ID: 249040206

Project Supervisor: Dr. Martina Palusa
Principal Marker: Dr. Anand Sengodan

Word Count: 1472
26/06/2025

DECLARATION

All sentences or passages quoted in this report, or computer code of any form whatsoever used and/or submitted at any stages, which are taken from other people's work have been specifically acknowledged by clear citation of the source, specifying author, work, date and page(s). Any part of my own written work, or software coding, which is substantially based upon other people's work, is duly accompanied by clear citation of the source, specifying author, work, date and page(s). I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this module and the degree examination as a whole.

Name: Parth Maru
Date: 26/06/2025

Contents

1. Aims and Objectives 3
2. Requirements 4
3. Technical Specification 4
4. Requirements Evaluation Plan 5
5. Background Research and Reading list 5
6. Time-plan and Risk Plan 7
7. References 8

1. Aims and Objectives

The primary aim of this project is to design and develop a secure, user-friendly, and modular web-based **bug tracking system** that facilitates the effective reporting, monitoring, and resolution of software issues by different stakeholders such as developers, testers, and administrators.

This system will support core features such as user authentication, role-based access (e.g., developer, reporter, admin), ticket lifecycle management, and detailed reporting.

By building the system from scratch using **Spring Boot (backend)** and **React + Tailwind (frontend)** with **PostgreSQL** as the primary data store, this project seeks to demonstrate practical application of modern full-stack development practices while maintaining clean architecture, scalability, and extensibility.

The platform is targeted for small to mid-sized development teams or open-source projects looking for a minimal but extendable alternative to tools like Jira or Bugzilla.

To accomplish the aim of building this system, the project will meet the following key objectives:

1. **Requirement Analysis & Planning**
    a. Identify roles (e.g., developer, reporter, admin) and define their capabilities.
    b. Design minimal yet scalable feature set for the MVP.
    c. Plan deliverables around university timelines.
2. **Frontend Development using React + Tailwind**
    a. Build a responsive landing page and login/register pages.
    b. Implement protected routes and token handling post-login.
    c. Design intuitive UI for ticket creation, tracking, and status updates.
3. **Backend Development using Spring Boot**
    a. Create RESTful APIs for user registration, authentication, and bug operations.
    b. Secure endpoints using JWT-based authentication.
    c. Implement logging, error handling, and input validation.
4. **Database & Schema Design**
    a. Design normalized PostgreSQL schema for users, bugs, roles, status, comments, etc.
    b. Use Flyway for database migrations and versioning.
5. **Authentication & Authorization**
    a. Register and authenticate users using password hashing and JWT.
    b. Implement role-based access control in the backend.
    c. Integrate with frontend for token-based session flow.
6. **Testing & Debugging**
    a. Perform unit and integration testing on backend APIs.
    b. Test frontend-to-backend integration using Postman and browser.
    c. Identify and resolve critical edge cases and authorization issues.
7. **Project Management and Version Control**

   a. Maintain feature branches and Git commit discipline.
   b. Follow structured GitLab issue and merge request workflow.
8. **Documentation**
   a. Document system architecture, APIs, and module interactions.
   b. Maintain README.md, setup guides, and development notes.
9. **Report Writing & Presentations**
   a. Submit interim and final reports with visual artifacts (UML diagrams, screenshots, etc.).
   b. Present working modules to supervisor in two-week intervals.

2. Requirements

- **Essential**
  - Secure user authentication (JWT-based)
  - Role-based access: Developer, Reporter, Admin
  - Bug submission and management features
  - Backend built with Spring Boot and PostgreSQL
  - Frontend with React, TypeScript, Tailwind CSS
  - Project/team creation flows
- **Recommended**
  - Notification system for bug assignment through email
  - Real-time updates using WebSocket
  - Team/ User based dashboards
- **Optional**
  - AI/ keywords-based bug deduplication (planned from August, using AI is optional)
  - Email alert integrations
  - Audit logging system

3. Technical Specification

- **Frontend**: React + TypeScript + Tailwind CSS + Shadcn UI
- **Backend**: Spring Boot (Java 17)
- **Database**: PostgreSQL
- **Authentication**: JWT token-based with stateless sessions
- **ORM/Migration**: JPA (Hibernate) + Flyway
- **Logging**: SLF4J with Spring Boot default logging
- **Dev Tools**: IntelliJ IDEA, Cursor IDE, Postman, Docker (planned but not confirmed), Git

4. Requirements Evaluation Plan

Evaluation Strategy:

- Unit testing with JUnit and Mockito for backend logic
- Manual UI testing using Postman and browser for end-to-end validation
- Functional testing with realistic workflows
- Supervisor walkthrough every 2 weeks simulating end-user actions
- Final presentation to demonstrate all MVP and advanced features

Success Criteria:

- User can register, login, and perform actions based on role
- Bugs can be created, assigned, updated, and tracked
- UI must remain responsive, clean, and intuitive
- Must pass security tests (invalid tokens, unauthorized access)
- Duplicate issues can be categorized, or at-least similar issues are listed with the help of keywords detection.

5. Background Research and Reading list

- **JWT Authentication – Concept and Specification [1]**
  JSON Web Tokens (JWT) are a widely adopted standard for securely transmitting information between parties. I studied the official JWT specifications and practical implementations to understand how tokens are signed, verified, and used for stateless authentication. This laid the foundation for implementing secure login mechanisms in the backend.
- **Spring Security Best Practices [2]**
  This resource provided a practical guide to configuring Spring Security for modern web applications. It covered topics such as authentication filters, stateless sessions, securing REST endpoints, CSRF protection, and the correct use of Authentication Manager — all essential for the security of the user authentication module.
- **PostgreSQL with Flyway Migrations [3] [4]**
  I explored how Flyway integrates schema versioning with PostgreSQL, enabling version-controlled migrations. This ensured a smooth setup of database tables and versioning for managing changes in the database structure during development, following industry practices.
- **UI/UX Minimalism in Web Design [5] [6] [7]**
  I referred to articles and visual design guidelines that emphasize clean, minimal interfaces. This helped shape the frontend layout with a strong focus on usability, whitespace, contrast, and distraction-free workflows. These ideas are implemented in the login and registration screens.

- **Full-stack Architecture Patterns**
  Studying typical full-stack patterns like client-server separation, API gateway practices, and service-oriented design helped structure the project into clearly separated React frontend and Spring Boot backend modules with RESTful interaction and token-based communication.
- **React Documentation [8]**
  Official React docs were referenced for understanding the component lifecycle, hooks (useState, useEffect), routing, and event-driven UI design. This was critical in implementing reusable and responsive UI components.
- **Tailwind CSS Docs [9]**
  I used Tailwind's utility-first CSS framework for styling. The documentation helped in composing layouts, responsive design, and achieving a consistent visual style using predefined class utilities for padding, spacing, font sizes, and colours.
- **Spring Boot Documentation [10]**
  I referred to the official docs to understand project structure, dependency management using Gradle, REST controller design, exception handling, and service annotations. These insights guided the creation of robust backend logic.

6. Time-plan and Risk Plan

A detailed timetable and plan for achieving the objectives of the project (this could be tabulated or in the form of a Gantt chart), including the milestones of the project and a risk plan; Explain target dates and amount of time required for the completion of aspects of the project

Development time plan:

Date	Milestone Description	Status
8 June – 22 June	Set up project, implement user authentication (JWT), login flow (UI + backend)	Completed
23 June – 5 July	Build user registration UI, validate and test frontend-backend integration	Ongoing
6 July – 15 July	MVP: Project/Team/Bug modules (basic CRUD)	Planned
16 July – 25 July	Finalize MVP, conduct polish, submit interim report	Planned
26 July – 31 July	Thorough testing of MVP	Planned
1 Aug – 15 Aug	Begin advanced features: duplicate detection, analytics, filtering, search	Planned
16 Aug – 29 Aug	System testing, frontend cleanup, advanced enhancements	Planned
30 Aug – 5 Sept	Final bug fixes, polish, submission prep	Planned

Risk plan:

Risk Involved	Mitigation strategy
Scope creep with too many advanced ideas	Prioritize MVP. Push complex features post-July
Integration delays between frontend/backend	Use component-based design and local mocks to parallelize work
Data loss or repo corruption	Maintain GitLab commits and Notion documentation
Deadline slippage	Weekly check-ins and supervisor feedback loops
Security bugs in JWT/Auth flow	Thorough testing with invalid/malformed tokens, strict role-based rules

7. References

[1] JWT.io Documentation – https://jwt.io/introduction

[2] Effective Spring Security – Baeldung Articles: https://www.baeldung.com/security-spring

[3] PostgreSQL Docs – https://www.postgresql.org/docs/

[4] Flyway Migration Tool – https://flywaydb.org/documentation/

[5] The Art of Simplifying: Minimalism in UI/UX Design - https://medium.com/%40MobileAppDesigner/the-art-of-simplifying-minimalism-in-ui-ux-design-25951aae457b

[6] The Art of Minimalism in UI/UX Design: Less is More - https://medium.com/%40brahimelmaghraoui56/the-art-of-minimalism-in-ui-ux-design-less-is-more-aba1fc4bd197

[7] Minimalism in UX/UI: Simplify your design without sacrificing functionality - https://medium.com/%40rave58120/minimalism-in-ux-ui-simplify-your-design-without-sacrificing-functionality-1ab888bc726b

[8] React Dev Docs – https://react.dev

[9] Tailwind CSS – https://tailwindcss.com/docs

[10] Spring Boot Documentation – https://docs.spring.io/spring-boot